STANDARD OPERATING PROCEDURE (SOP) | PRODUCTION BLUEPRINT

Headless WordPress + Astro Marketing Site

Full End-to-End Setup, Architecture & Vercel Deployment Guide

Doc ID: SOP-GRM-2026-01 | Reference: Good Rep Media | Stack: WPGraphQL + Gravity Forms + Astro 7 + React 19 + Vercel

Table of Contents

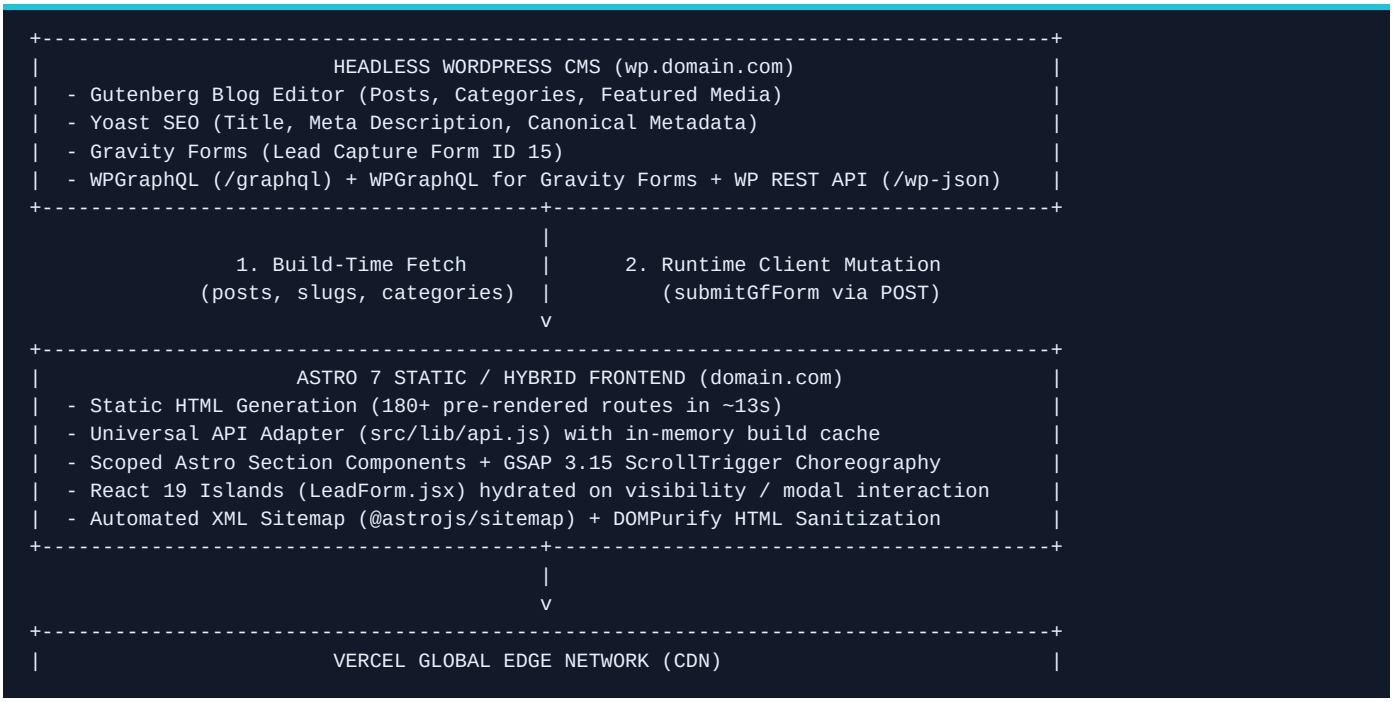
1. Executive Summary & System Architecture
2. Phase 1: Headless WordPress Setup, Plugins & PHP Snippets
3. Phase 2: Astro Project Initialization & Configuration
4. Phase 3: Folder Structure & Data-Driven Content Architecture
5. Phase 4: Dual-Mode WordPress API Layer & Security Sanitization
6. Phase 5: Component Splits, Master Templates & Design Rules
7. Phase 6: Gravity Forms React Island (LeadForm.jsx)
8. Phase 7: Vercel Deployment, DNS Cutover & Rebuild Webhooks
9. Phase 8: New Site Replication & QA Checklist

1. Executive Summary & System Architecture

1.1 Purpose

This Standard Operating Procedure (SOP) provides a complete, beginning-to-end engineering blueprint for building, migrating, and deploying high-converting agency and B2B/B2C marketing websites using Astro as a static/hybrid frontend and Headless WordPress as the content and lead-capture backend.

1.2 End-to-End Architecture Topology



1.3 Core Technology Stack & Version Matrix

Layer	Package / Tool	Version	Purpose
Runtime	Node.js	>= 22.12.0	Build environment & ES module execution
Framework	astro	^7.3.8	Static site generation (SSG), routing, scoped CSS
UI Islands	react / react-dom	^19.3.1	Interactive client-side lead forms (@astrojs/react)
Animation	gsap	^3.15.0	ScrollTrigger parallax, counters, & floating graphics
Security	isomorphic-dompurify	^3.19.0	XSS sanitization for inbound WordPress HTML & form confirmations
SEO	@astrojs/sitemap	^3.7.4	Automated sitemap-index.xml & sitemap-0.xml generation
Icons	material-symbols	^0.40.2	Google Material Symbols Outlined icon font
CMS Backend	WordPress + PHP	6.x / PHP 8.2+	Headless content authoring & Gravity Forms storage

2. Phase 1: Headless WordPress Setup, Plugins & PHP Snippets

2.1 Subdomain Strategy

When decoupling an existing WordPress site where WordPress currently occupies the apex domain (https://domain.com):

1. Staging / Prototype Phase: Keep WordPress at https://domain.com and connect Astro to https://domain.com/wp-json/wp/v2 (or /graphql).
2. Production Cutover Phase: Move WordPress to a dedicated subdomain (https://wp.domain.com) pointing to the WordPress hosting server IP (e.g., 23.111.181.43), freeing the apex domain (https://domain.com) and www for Vercel.

2.2 Required WordPress Plugins

Install and activate the following plugins in the WordPress admin dashboard (/wp-admin/plugins.php):

1. WPGraphQL (wp-graphql)
   - Exposes the /graphql endpoint for querying posts, categories, media attachments, and pages.
2. Gravity Forms (gravityforms)
   - Commercial form plugin used to store leads, trigger admin email notifications, and route CRM webhooks.
3. WPGraphQL for Gravity Forms (wp-graphql-gravity-forms)
   - Exposes the submitGfForm GraphQL mutation so the React LeadForm.jsx component can submit entries directly from the browser.
4. Yoast SEO (wordpress-seo) + WPGraphQL Yoast SEO Addon (add-wpgraphql-seo)
   - Exposes seo { title metaDesc } on GraphQL post nodes and yoast_head_json on REST API responses.
5. WP Webhooks (wp-webhooks) *(or custom save_post hook)*
   - Fires an HTTP POST request to a Vercel Deploy Hook URL whenever a blog post is published or updated.

2.3 Mandatory WordPress PHP Snippets

Create a must-use plugin at wp-content/mu-plugins/grm-headless-config.php (or append to the active theme's functions.php).

Snippet 1: Increase WPGraphQL Max Query Limit to 500

By default, WPGraphQL caps connection queries at 100 items. Raising this limit to 500 allows Astro static builds to fetch the entire blog archive in a single GraphQL request:

```php
<?php
/**
 * Plugin Name: GRM Headless WordPress Configuration
 * Description: Configures WPGraphQL query limits, CORS headers, and Vercel webhooks for Astro.
 */

// 1. Increase WPGraphQL max query limit for Astro static builds
add_filter( 'graphql_connection_max_query_amount', function( $amount, $source, $args, $context, $info ) {
    return 500;
}, 10, 5 );
```

Snippet 2: Configure CORS Headers for Browser GraphQL Submissions
Because LeadForm.jsx sends a client-side POST request from https://domain.com (or http://localhost:4321 during local development) to https://wp.domain.com/graphql, WordPress must return permissive CORS headers for /graphql:

```php
// 2. Allow CORS on WPGraphQL endpoint for frontend LeadForm submissions
add_filter( 'graphql_response_headers_to_send', function( $headers ) {
    $allowed_origins = [
        'https://goodrep.media',
        'https://www.goodrep.media',
        'https://jgyb89.github.io',
        'http://localhost:4321',
    ];

    $origin = isset( $_SERVER['HTTP_ORIGIN'] ) ? $_SERVER['HTTP_ORIGIN'] : '';
    if ( in_array( $origin, $allowed_origins, true ) ) {
        $headers['Access-Control-Allow-Origin']      = $origin;
        $headers['Access-Control-Allow-Methods']     = 'POST, GET, OPTIONS';
        $headers['Access-Control-Allow-Headers']     = 'Content-Type, Authorization';
        $headers['Access-Control-Allow-Credentials'] = 'true';
    }
    return $headers;
} );
```

Snippet 3: Automated Vercel Deploy Hook on Post Publish/Update
Automatically trigger a fresh Astro static build on Vercel whenever an editor publishes or updates a post:

```php
// 3. Trigger Vercel Deploy Hook when a published post is created or updated
add_action( 'save_post_post', function( $post_id, $post, $update ) {
    if ( wp_is_post_revision( $post_id ) || wp_is_post_autosave( $post_id ) ) {
        return;
    }
    if ( $post->post_status !== 'publish' ) {
        return;
    }

    $vercel_hook_url = defined( 'GRM_VERCEL_DEPLOY_HOOK' ) ? GRM_VERCEL_DEPLOY_HOOK : '';
    if ( ! empty( $vercel_hook_url ) ) {
        wp_remote_post( $vercel_hook_url, [ 'timeout' => 5, 'blocking' => false ] );
    }
}, 10, 3 );
```

2.4 Wordfence WAF & Firewall Whitelisting

If Wordfence Security or a host-level WAF (Cloudflare / LiteSpeed ModSecurity) is active on the WordPress server:

1. Navigate to Wordfence -> All Options -> Whitelisted URLs.
2. Add /graphql for POST requests so anonymous client-side submitGfForm mutations are not blocked with HTTP 403 Forbidden.
3. Under WPGraphQL -> Settings, ensure Public Introspection or public query execution is enabled for published posts and Gravity Forms mutations.

2.5 Gravity Forms Field ID Mapping

In Forms -> New Form, create the Consultation Lead Form (Form ID 15 in the reference implementation) with the following field IDs mapped to GF_FIELD_IDS in src/components/LeadForm.jsx:

Field Label	Gravity Forms Field Type	Field ID	GraphQL Input Key	Required
First Name	Single Line Text	12	value: String	Yes
Last Name	Single Line Text	13	value: String	Yes
Email	Email	5	emailValues: { value: String }	Yes
Phone	Phone (Standard)	6	value: String	Yes
Company / Org	Single Line Text	4	value: String	No
Services Needed	Single Line Text / Checkboxes	7	value: String (comma-joined)	Yes
Message	Paragraph Text	9	value: String	No

3. Phase 2: Astro Project Initialization & Configuration

3.1 Project Bootstrap & Dependencies

To initialize a new marketing site following this architecture:

```bash
# 1. Create a new Astro project
npm create astro@latest my-marketing-frontend -- --template minimal
cd my-marketing-frontend

# 2. Install core runtime dependencies
npm install @astrojs/react @astrojs/sitemap react react-dom gsap isomorphic-dompurify material-symbols

# 3. Install TypeScript React definitions
npm install -D @types/react @types/react-dom
```

3.2 package.json Configuration

Ensure Node >=22.12.0 is specified along with standard build and preview scripts:

```json
{
  "name": "goodrep-media-frontend",
  "type": "module",
  "version": "0.0.1",
  "engines": {
    "node": ">=22.12.0"
  },
  "scripts": {
    "dev": "astro dev",
    "build": "astro build",
    "build:gh-pages": "GITHUB_PAGES=true astro build && node scripts/prepare-gh-pages.mjs",
    "preview": "astro preview",
    "astro": "astro"
  },
  "dependencies": {
    "@astrojs/react": "^7.0.1",
    "@astrojs/sitemap": "^3.7.4",
    "@types/react": "^19.3.0",
    "@types/react-dom": "^19.3.0",
    "astro": "^7.3.8",
    "gsap": "^3.15.0",
    "isomorphic-dompurify": "^3.19.0",
    "material-symbols": "^0.40.2",
    "react": "^19.3.1",
    "react-dom": "^19.3.1"
  }
}
```

3.3 astro.config.mjs Setup

Configure @astrojs/react for interactive form islands and @astrojs/sitemap for automated XML sitemaps:

```javascript
import { defineConfig } from 'astro/config';
import react from '@astrojs/react';
import sitemap from '@astrojs/sitemap';

const isGitHubPages = process.env.GITHUB_PAGES === 'true';

export default defineConfig({
  site: isGitHubPages ? 'https://jgyb89.github.io' : 'https://goodrep.media',
  base: isGitHubPages ? '/goodrep-media-frontend' : '/',
  integrations: [react(), sitemap()],
});
```

3.4 Environment Variables (.env)

Create a .env file in the project root:

```ini
# Prototype / Migration Mode (WP REST API)
PUBLIC_WORDPRESS_API_URL=https://goodrep.media/wp-json/wp/v2

# Production Mode (WPGraphQL on dedicated subdomain)
# PUBLIC_WORDPRESS_API_URL=https://wp.goodrep.media/graphql
# PUBLIC_WORDPRESS_GRAPHQL_URL=https://wp.goodrep.media/graphql
```

4. Phase 3: Folder Structure & Data-Driven Content Architecture

4.1 Complete Project Directory Tree

```text
goodrep-media-frontend/
+-- .env                              # Environment variables (API endpoints)
+-- .github/workflows/deploy.yml      # Optional GitHub Pages CI/CD workflow
+-- astro.config.mjs                  # Astro configuration (React, Sitemap, site URL)
+-- package.json                      # Dependencies & scripts
+-- scripts/
|   `-- prepare-gh-pages.mjs          # Base-path rewriter for subpath staging deployments
+-- public/                           # Static binary assets served at root "/"
|   +-- fonts/                        # Self-hosted brand fonts (if applicable)
|   +-- grm-case-study/               # Portfolio case study photography & screenshots
|   +-- grm-graphic-samples/          # Client logos & graphic design showcase items
|   +-- grm-landing-pages/            # Industry landing page hero WebP graphics
|   +-- grm-main-graphics/            # Primary 3D centerpiece illustrations (1 per stage)
|   +-- grm-web-assets/               # Assisting floating badges, blobs & satellites
|   `-- grm-web-icons/                # UI icons, step number badges & pillar icons
`-- src/
    +-- config/
    |   `-- site.js                   # Single source of truth for phone, email, legal links
    +-- styles/
    |   +-- global.css                # Design tokens, typography, buttons, container grid
    |   `-- gutenberg.css             # Scoped WordPress Gutenberg block styles for blog posts
    +-- lib/
    |   +-- api.js                    # Universal WPGraphQL + REST API adapter & build cache
    |   `-- animations.js             # Shared GSAP ScrollTrigger & floating animation helpers
    +-- layouts/
    |   +-- MainLayout.astro          # Global header, mega-menu, footer & lead modal
    |   `-- LandingLayout.astro       # Focused conversion layout for paid ad landing pages
    +-- components/
    |   +-- BaseHead.astro            # Meta tags, canonical URL, OpenGraph, fonts
    |   +-- ServiceHero.astro         # Master hero for all /services/* pages
    |   +-- StartsSection.astro       # 2-column feature spotlight + animated SVG ring callout
    |   +-- StatsBand.astro           # Dark animated number counter band
    |   +-- ProcessSection.astro      # Interactive accordion + 4-step synced visual stage
    |   +-- PillarsSection.astro      # Numbered pillar cards + parallax graphic stage
    |   +-- DeliverablesGrid.astro    # 3-column deliverables grid with floating accents
    |   +-- ReadyBand.astro           # Dark #1a1a1a CTA band extending behind consult waves
    |   +-- ConsultSection.astro      # Animated SVG wave ribbon + LeadForm section
    |   +-- CaseStudyHero.astro       # Master hero for /work/[slug] case studies
    |   +-- CaseStudyStats.astro      # KPI results strip for case studies
    |   +-- CaseStudyNarrative.astro  # Challenge / Solution / Results narrative layout
    |   +-- LightboxModal.astro       # Fullscreen image gallery lightbox
    |   +-- LeadForm.jsx              # React 19 Gravity Forms component
    |   +-- LeadForm.css              # Form layout, validation error states & pill checkboxes
    |   `-- landing/
    |       `-- FaqSection.astro      # Reusable FAQ accordion for landing pages
    +-- data/
    |   +-- homeData.js               # Homepage hero slides, services grid & process steps
    |   +-- webDesignData.js          # Content model for /services/web-design (and 11 others)
    |   +-- lawnCareData.js           # Content model for /lawn-care-business (and 2 others)
    |   `-- work/
    |       +-- caseStudiesRegistry.js # Central route map for all case studies & legacy aliases
    |       +-- directoryData.js      # Portfolio hub filtering & card metadata
    |       `-- *.js                  # Individual case study data models (10 files)
    `-- pages/
```

```
        +-- index.astro              # Homepage
        +-- about.astro              # About page
        +-- services.astro           # Services hub page
        +-- services/
        |   +-- [alias].astro        # Dynamic route generator for legacy WP service URLs
        |   `-- *.astro              # 12 canonical service detail pages
        +-- work.astro               # Portfolio showcase directory
        +-- work/[slug].astro        # Canonical case study template
        +-- our-work/[slug].astro    # Legacy URL alias template for case studies
        +-- blog/
        |   +-- index.astro          # Knowledge Center archive with category filtering
        |   `-- [slug].astro         # Dynamic blog article template (127+ static pages)
        +-- lawn-care-business.astro  # Industry landing page
        +-- medspa-google-ads.astro   # Industry landing page
        `-- spa-esthetician-business.astro # Industry landing page
```

4.2 Separation of Content Data (src/data/*.js) vs Presentation (src/components/*.astro)

A core architectural principle of this project is zero inline CSS styles (style="...") in data files:

- Data files (src/data/*.js) store pure content models: text copy, CTA links, image paths (src, alt), parallax depth numbers, and semantic Directional Placement tokens (placement: "top-left" | "top-right" | "bottom-left" | "bottom-right").
- Astro components (src/components/*.astro) own all layout coordinates, responsive widths, z-index stacking, and default intrinsic dimensions (width={sat.width ?? 746}, height={sat.height ?? 780}).
- Why this matters: In the browser DOM, element.style is reserved exclusively for active GSAP runtime transforms (translate3d, rotate), preventing GSAP's clearProps from wiping out static positioning.

Example Data Model (src/data/webDesignData.js)

JAVASCRIPT

```javascript
export const startsData = {
  eyebrow: "YOUR 24/7 DIGITAL STOREFRONT",
  title: "It All Starts With Your Website",
  question: "Do your customers love your website, or tolerate it?",
  body: [
    "Your website can make or break you- and that's not an exaggeration...",
  ],
  callout: {
    percent: 80,
    title: "IS YOUR SITE MOBILE READY?",
    text: "<strong>80% of people</strong> who have made purchases online in the last year did so on a smart...
  },
  media: {
    blob: {
      src: "/grm-web-assets/blue-circular-background-2-graphic.svg",
    },
    main: {
      src: "/grm-main-graphics/laptop-web-design-layout-graphic.svg",
      alt: "Responsive laptop and mobile website experience",
    },
    satellites: [
      {
        src: "/grm-web-assets/3d-ruler-graphic.svg",
        parallax: -36,
        placement: "top-left",
      },
      {
        src: "/grm-web-assets/77-percent-bubble-graphic.svg",
        parallax: -44,
```

```
        placement: "bottom-right",
      },
    ],
    pill: {
      icon: "smartphone",
      title: "80% Mobile Traffic",
      subtitle: "Responsive mobile-first architectures",
      parallax: -30,
    },
  },
};
```

5. Phase 4: Dual-Mode WordPress API Layer & Security Sanitization

5.1 Universal API Adapter (src/lib/api.js)

src/lib/api.js is architected to support both WordPress REST API (/wp-json/wp/v2) and WPGraphQL (/graphql) automatically based on PUBLIC_WORDPRESS_API_URL:

- If PUBLIC_WORDPRESS_API_URL ends with /graphql, isGraphQL is true and queries execute via GraphQL POST.
- Otherwise, src/lib/api.js seamlessly falls back to the WordPress REST API and normalizes every post into the exact same object shape.
- Single-Fetch Build Cache (ensureFullPostsCache()): During astro build, all published posts are fetched once, normalized, and stored in an in-memory Map (cachedPostBySlug). Generating 127+ static blog pages takes only 1 network request (or 2 paginated REST requests) instead of 127 individual requests.

JAVASCRIPT

```
const API_URL = (
  (typeof import.meta !== "undefined" && import.meta.env?.PUBLIC_WORDPRESS_API_URL) ||
  (typeof process !== "undefined" && process.env?.PUBLIC_WORDPRESS_API_URL) ||
  "https://goodrep.media/wp-json/wp/v2"
).replace(/\/$/, "");

const isGraphQL = API_URL.endsWith("/graphql");
```

5.2 GraphQL Query for Static Builds (posts(first: 500))

When isGraphQL is active, ensureFullPostsCache() executes:

GRAPHQL

```
query GetAllPostsForStaticBuild {
  posts(first: 500, where: { status: PUBLISH }) {
    nodes {
      id
      databaseId
      title
      slug
      date
      excerpt
      content
      featuredImage {
        node {
          sourceUrl
          altText
        }
      }
```

```
      categories {
        nodes {
          databaseId
          name
          slug
        }
      }
      seo {
        title
        metaDesc
      }
    }
  }
}
```

5.3 XSS HTML Sanitization (isomorphic-dompurify)

Never render raw CMS HTML without sanitization. In src/pages/blog/[slug].astro, all inbound WordPress content is scrubbed at build time before passing to set:html:

```astro
---
import DOMPurify from "isomorphic-dompurify";
import { getAllPostSlugs, getPostBySlug } from "../../lib/api.js";

export async function getStaticPaths() {
  const slugs = await getAllPostSlugs();
  return slugs.map(({ slug }) => ({ params: { slug } }));
}

const { slug } = Astro.params;
const post = await getPostBySlug(slug);
const cleanContent = DOMPurify.sanitize(post?.content || "");
---
<article class="gutenberg-content" set:html={cleanContent} />
```

6. Phase 5: Component Splits, Master Templates & Design Rules

6.1 Master Page Templates

Every group of same-type pages shares a unified component composition so branding, spacing, and animations remain 100% consistent:

1. Service Detail Pages (src/pages/services/*.astro):
   - <ServiceHero {...heroData} />
   - <StartsSection {...startsData} />
   - <StatsBand {...statsData} />
   - <ProcessSection {...processData} />
   - <PillarsSection {...pillarsData} /> *(or secondary <StartsSection>)*
   - <DeliverablesGrid {...deliverablesData} />
   - <ReadyBand {...readyData} />
   - <ConsultSection {...consultData} />
2. Case Study Pages (src/pages/work/[slug].astro):
   - <CaseStudyHero /> -> <CaseStudyStats /> -> <CaseStudyNarrative /> -> <LightboxModal /> -> <ConsultSection />

3. Paid Ad Landing Pages (src/pages/lawn-care-business.astro, etc.):
   - Wrapped in <LandingLayout /> -> Industry Hero -> Pillars -> Marquee -> Why Choose Us -> Metrics -> Deliverables -> Case Study Chapters -> <FaqSection /> -> <ConsultSection />

6.2 Route-Scoped Consultation Wave & ReadyBand Integration

A signature visual feature of the /services pages is the transition from the dark #1a1a1a ReadyBand section into the cyan ConsultSection:

- On /services and /services/* routes:
  - ReadyBand.astro extends .ready-band__surface downward by the exact wave ribbon height (padding-bottom: clamp(55px, 8.5vw, 115px); margin-bottom: calc(-1 * clamp(55px, 8.5vw, 115px));).
  - ConsultSection.astro detects Astro.url.pathname.includes("/services") (waveTheme = "dark") and applies .consult--services and .consult__wave--dark.
  - The wave SVG background is transparent and the animated wave paths are anchored to the bottom edge (M0,120 L1440,120 ...) with color: var(--color-cyan), so the moving cyan waves rise directly over the extended dark patterned ready-band__surface behind them.
- On all other pages outside /services:
  - waveTheme = "light" is used. The wave SVG masks the top edge (M0,0 L1440,0 ...) in white (var(--color-bg)), seamlessly blending into the preceding white section above the cyan waves.

```css
/* ReadyBand.astro: Extend dark surface behind ConsultSection wave ribbon */
.ready-band__surface {
  padding-top: clamp(2rem, 5vw, 4rem);
  padding-bottom: clamp(55px, 8.5vw, 115px);
  margin-bottom: calc(-1 * clamp(55px, 8.5vw, 115px));
}

/* ConsultSection.astro: Transparent wave zone on /services routes */
.consult--services {
  --consult-wave-h: clamp(55px, 8.5vw, 115px);
  background: transparent;
  z-index: 2;
}
.consult--services::before {
  content: "";
  position: absolute;
  top: calc(var(--consult-wave-h) - 2px);
  inset-inline: 0;
  bottom: 0;
  background: var(--color-cyan);
  z-index: -1;
}
.consult__wave--dark {
  top: 0;
  background: transparent;
  color: var(--color-cyan);
  transform-origin: center bottom;
}
```

6.3 Strict Visual & Motion Engineering Rules

1. Zero Text Occlusion: Graphical elements, floating badges, and decorative assets must never overlap or obscure headings, body copy, or buttons at any viewport width.
2. Anti-Clutter Graphic Formula: Every media stage (outside the homepage hero) consists strictly of:
   - 1 Centerpiece Graphic (public/grm-main-graphics/)
   - 1 Background Blob (blue-circular-background-graphic.svg)

- At most 2 Opposite-Quadrant Floating Satellites (top-left + bottom-right or bottom-left + top-right)

3. GSAP Motion Hygiene:
   - Wrap all scroll/float animations in gsap.matchMedia().add("(prefers-reduced-motion: no-preference)", () => { ... }).
   - Always include clearProps: "transform,opacity" on entrance tweens ([data-reveal]) so elements return to clean CSS state after revealing.

7. Phase 6: Gravity Forms React Island (LeadForm.jsx)

src/components/LeadForm.jsx is a self-contained React 19 component rendered in two places:

1. Inside ConsultSection.astro using client:visible (hydrates only when the user scrolls near the footer).
2. Inside the global popup modal in MainLayout.astro (hydrates lazily when any [data-open-modal] button is clicked).

Key Implementation Details:

- Multi-Instance Safe IDs: Uses React's useId() hook (${uid}-firstName, etc.) so multiple forms on the same page never have colliding HTML id attributes.
- US Phone Auto-Masking: Automatically formats phone input as (XXX) XXX-XXXX and strips leading +1 country codes on paste.
- Strict WPGraphQL Gravity Forms Mutation:
  - Regular text fields pass { id: FIELD_ID, value: "..." }.
  - Email field (id: 5) passes { id: 5, emailValues: { value: "..." } }.
  - Confirmation HTML returned by Gravity Forms is sanitized with DOMPurify.sanitize(confirmationMessage) before rendering.

```javascript
export const GF_FIELD_IDS = Object.freeze({
  COMPANY: 4,
  EMAIL: 5,
  PHONE: 6,
  SERVICES_NEEDED: 7,
  MESSAGE: 9,
  FIRST_NAME: 12,
  LAST_NAME: 13,
});

const SUBMIT_GF_FORM_MUTATION = `
  mutation SubmitLeadForm($input: SubmitGfFormInput!) {
    submitGfForm(input: $input) {
      confirmation { type message url }
      errors { id message }
    }
  }
`;

const fieldValues = [
  { id: GF_FIELD_IDS.FIRST_NAME, value: formData.firstName || '' },
  { id: GF_FIELD_IDS.LAST_NAME, value: formData.lastName || '' },
  { id: GF_FIELD_IDS.EMAIL, emailValues: { value: formData.email || '' } },
  { id: GF_FIELD_IDS.PHONE, value: formData.phone || '' },
  { id: GF_FIELD_IDS.COMPANY, value: formData.company || '' },
  { id: GF_FIELD_IDS.SERVICES_NEEDED, value: multipleChoiceValue },
  { id: GF_FIELD_IDS.MESSAGE, value: formData.message || '' },
];
```

8. Phase 7: Vercel Deployment, DNS Cutover & Rebuild Webhooks

8.1 Step-by-Step Vercel Setup

1. Push the repository to GitHub (main branch).
2. Log in to Vercel -> Add New... -> Project -> Import the GitHub repository.
3. Verify the Build and Output Settings (auto-detected by Vercel):
   - Framework Preset: Astro
   - Build Command: astro build (or npm run build)
   - Output Directory: dist
   - Install Command: npm install
   - Node.js Version: 22.x (in Project Settings -> General)
4. Under Environment Variables, add:
   - PUBLIC_WORDPRESS_API_URL = https://wp.goodrep.media/graphql *(or https://goodrep.media/wp-json/wp/v2 prior to subdomain cutover)*
   - PUBLIC_WORDPRESS_GRAPHQL_URL = https://wp.goodrep.media/graphql
5. Click Deploy. Vercel will build all 180+ static pages in ~15 seconds and assign a .vercel.app preview URL.

8.2 DNS & Domain Cutover Checklist

When ready to switch production traffic to Vercel while retaining WordPress on wp.domain.com:

Record Type	Host / Name	Value / Target	TTL	Purpose
A	wp	23.111.181.43 *(WP Server IP)*	300	Hosts Headless WordPress & /graphql API
A	@	76.76.21.21	300	Points apex domain (domain.com) to Vercel
CNAME	www	cname.vercel-dns.com	300	Points www.domain.com to Vercel

8.3 Configuring Vercel Deploy Hooks for WordPress

1. In Vercel, navigate to Project Settings -> Git -> Deploy Hooks.
2. Create a hook named WordPress Content Publish on branch main.
3. Copy the generated URL (https://api.vercel.com/v1/integrations/deploy/prj_.../xxx).
4. Add define('GRM_VERCEL_DEPLOY_HOOK', 'https://api.vercel.com/v1/integrations/deploy/...'); to wp-config.php (used by Snippet 3 in Section 2.3) or paste it into WP Webhooks -> Send Data -> Post Updated / Post Created.

9. Phase 8: New Site Replication & QA Checklist

Use this checklist when duplicating this architecture for a new client or brand:

[ ] 1. Brand & Global Config (src/config/site.js & src/styles/global.css):
- Update SITE_NAME, CONTACT_INFO (phone, email), LEGAL_LINKS, and CSS custom properties (--color-primary, --color-cyan, --color-dark, --font-heading, --font-body).

[ ] 2. WordPress Backend Verification:
- Confirm WPGraphQL, Gravity Forms, and WPGraphQL for Gravity Forms are active.
- Verify graphql_connection_max_query_amount filter (500) and CORS headers are active in mu-plugins.
- Match GF_FIELD_IDS in src/components/LeadForm.jsx to the target Gravity Form IDs.

[ ] 3. Content & Asset Population (src/data/*.js & public/):

- Populate service data files, landing page data files, and case study registry without inline style: strings (use placement: "top-left" | "top-right" | "bottom-left" | "bottom-right").

[ ] 4. Build & Sitemap Verification:

- Run npm run build locally and confirm 0 errors, dist/sitemap-index.xml generation, and clean HTML output.

[ ] 5. End-to-End Form & Mobile QA:

- Test LeadForm validation, phone masking, and live GraphQL submission on desktop and mobile viewports.
- Verify zero text occlusion across all hero, spotlight, process, and consultation sections.